Labsheet 7

1. Write a C++ program to create a Binary Tree using dynamically allocated nodes and display its structure.

```
int main76() {
    TNode* root = new TNode(1);
    root->left = new TNode(2);
    root->right = new TNode(3);

    cout << "Root = " << root->data << endl;
    cout << "Left child = " << root->left->data << endl;
    cout << "Right child = " << root->right->data << endl;

    return 0;
}
```

2. Write a C++ program to implement Inorder Traversal of a Binary Tree.

```
int main77() {
    TNode* root = new TNode(1);
    root->left = new TNode(2);
    root->right = new TNode(3);

    inorder(root);

    return 0;
}
```

3. Write a C++ program to implement Preorder Traversal of a Binary Tree.

```
int main78() {
    TNode* root = new TNode(1);
    root->left = new TNode(2);
    root->right = new TNode(3);

    preorder(root);

    return 0;
}
```

4. Write a C++ program to implement Postorder Traversal of a Binary Tree.

```
int main79() {
    TNode* root = new TNode(1);
    root->left = new TNode(2);
    root->right = new TNode(3);

    postorder(root);

    return 0;
}
```

5. Write a C++ program to implement all three traversals — Inorder, Preorder, and Postorder — for the same Binary Tree.

```cpp
int main80() {
    TNode* root = new TNode(1);
    root->left = new TNode(2);
    root->right = new TNode(3);

    cout << "Inorder: ";
    inorder(root);

    cout << "\nPreorder: ";
    preorder(root);

    cout << "\nPostorder: ";
    postorder(root);

    return 0;
}
```

6. Write a C++ program to create a Binary Search Tree (BST) by inserting n elements.

```cpp
int main81() {
    TNode* root = NULL;
    int n, value;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";

    for (int i = 0; i < n; i++) {
        cin >> value;
        root = insertBST(root, value);
    }

    cout << "BST Inorder: ";
    inorder(root);

    return 0;
}
```

7. Write a C++ program to perform searching in a Binary Search Tree for a given key.

```cpp
int main82() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    insertBST(root, 30);
    insertBST(root, 70);
    insertBST(root, 20);
    insertBST(root, 40);

    int key;
    cout << "Enter key: ";
    cin >> key;

    if (searchBST(root, key))
        cout << "Element found";
    else
        cout << "Element not found";

    return 0;
}
```

8. Write a C++ program to perform insertion in a Binary Search Tree and display its inorder traversal.

```
int main83() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    root = insertBST(root, 30);
    root = insertBST(root, 70);
    root = insertBST(root, 20);
    root = insertBST(root, 40);

    cout << "Inorder: ";
    inorder(root);

    return 0;
}
```

9. Write a C++ program to delete a leaf node from a Binary Search Tree.

```
int main84() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    root = insertBST(root, 30);
    root = insertBST(root, 70);

    root = deleteBST(root, 30);

    cout << "After deleting leaf: ";
    inorder(root);

    return 0;
}
```

10. Write a C++ program to delete a node having one child from a Binary Search Tree.

```
int main85() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    root = insertBST(root, 30);
    root = insertBST(root, 20);

    root = deleteBST(root, 30);

    cout << "After deletion: ";
    inorder(root);

    return 0;
}
```

11. Write a C++ program to delete a node having two children from a Binary Search Tree using the inorder successor.

```
int main86() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    root = insertBST(root, 30);
    root = insertBST(root, 70);
    root = insertBST(root, 20);
    root = insertBST(root, 40);
    root = insertBST(root, 60);
    root = insertBST(root, 80);

    root = deleteBST(root, 50);

    cout << "After deleting 50: ";
    inorder(root);

    return 0;
}
```

12. Write a C++ program to calculate the height of a Binary Tree.

```cpp
int main87() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    root = insertBST(root, 30);
    root = insertBST(root, 70);
    root = insertBST(root, 20);
    root = insertBST(root, 40);

    cout << "Height = " << treeHeight(root);

    return 0;
}
```

13. Write a C++ program to count the total number of nodes in a Binary Tree.

```cpp
int main88() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    root = insertBST(root, 30);
    root = insertBST(root, 70);

    cout << "Total nodes = " << countNodes(root);

    return 0;
}
```

14. Write a C++ program to count the number of leaf nodes in a Binary Tree.

```cpp
int main89() {
    TNode* root = NULL;

    root = insertBST(root, 50);
    root = insertBST(root, 30);
    root = insertBST(root, 70);
    root = insertBST(root, 20);
    root = insertBST(root, 40);

    cout << "Leaf nodes = " << countLeaves(root);

    return 0;
}
```

15. Write a C++ program to implement a menu-driven Binary Search Tree with the following operations: Insert, Search, Inorder Traversal, Preorder Traversal, Postorder Traversal, Delete, Height, Count Nodes, Exit.

```cpp
int main90() {
    TNode* root = NULL;
    int choice, value;

    do {
        cout << "\n--- BST MENU ---\n";
        cout << "1. Insert\n";
        cout << "2. Search\n";
        cout << "3. Inorder\n";
        cout << "4. Preorder\n";
        cout << "5. Postorder\n";
        cout << "6. Delete\n";
        cout << "7. Height\n";
        cout << "8. Count Nodes\n";
        cout << "9. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter value: ";
                cin >> value;
                root = insertBST(root, value);
                break;

            case 2:
                cout << "Enter value: ";
                cin >> value;

                if (searchBST(root, value))
                    cout << "Found\n";
                else
                    cout << "Not found\n";
                break;
```

```cpp
            case 3:
                cout << "Inorder: ";
                inorder(root);
                cout << endl;
                break;

            case 4:
                cout << "Preorder: ";
                preorder(root);
                cout << endl;
                break;

            case 5:
                cout << "Postorder: ";
                postorder(root);
                cout << endl;
                break;

            case 6:
                cout << "Enter value: ";
                cin >> value;
                root = deleteBST(root, value);
                break;

            case 7:
                cout << "Height = " << treeHeight(root) << endl;
                break;

            case 8:
                cout << "Total nodes = " << countNodes(root) << endl;
                break;

            case 9:
```

```cpp
                cout << "Program terminated\n";
                break;

            default:
                cout << "Invalid choice\n";
        }

    } while (choice != 9);

    return 0;
}
```